

Nitish Rathore

413-466-2322 | nitishrathore9@gmail.com | linkedin.com/in/nitish-vinay-rathore | github.com/Nit-Rathore

EDUCATION

University of Massachusetts Amherst

Sep. 2026 – May 2028 (Expected)

Master of Science in Computer Science

Amherst, MA

Coursework: Machine Learning, Theory & Practice of Software Engineering, Trustworthy & Responsible AI

PES University

Dec. 2020 – May 2024

Bachelor of Technology in Computer Science and Engineering

GPA: 8.66/10

Coursework: Data Structures & Algorithms, Database Systems, Operating Systems, Cloud Computing, Big Data

TECHNICAL SKILLS

Languages: Python, Java, C#, C/C++, JavaScript, TypeScript, SQL

Backend & APIs: ASP.NET Core, Ray Serve, REST APIs, Microservices, Event-Driven Architecture, Pytest

Data & Messaging: ClickHouse, MongoDB, Redis, RabbitMQ, Kafka, ZooKeeper, Pub/Sub Messaging

Cloud & DevOps: AWS (Lambda, SQS, Cognito, S3, EC2, ECS), Docker, Kubernetes, GitHub Actions, CI/CD, Git

Frontend & ML: React, Next.js, PyTorch, TensorFlow, OpenCV, ONNX Runtime

EXPERIENCE

KangaSys | *Software Engineer*

Mar. 2025 – Jun. 2026

- Built a multi-tenant IoT analytics platform in ASP.NET Core, ClickHouse, and AWS that ingests **10M+ daily sensor events** from Honeywell building management systems for real-time monitoring.
- Tuned ClickHouse aggregations and indexing to serve queries in **100–500 ms** over 1M+ record time-series tables.
- Developed Next.js 14 dashboards with **30+ configurable widgets**, drag-and-drop layouts, and AWS Cognito MFA.
- Implemented serverless async PDF/CSV report exports (AWS Lambda + SQS), tenant-level RBAC with Permit.io, and automated CI/CD pipelines to AWS.

Pronisi, Inc. | *Software Engineer*

Jun. 2024 – Feb. 2025

- Led the computer-vision engineering for **SlamDunk.AI**, an AI sports-coaching platform, architecting **8 Ray Serve microservices** (Tennis, Yoga, Dance, and Gym × preprocessing/recommendation stages) with independent REST routes and RabbitMQ-driven async messaging.
- Parallelized computer-vision inference by running pose, ball, racket, and court-baseline detection as concurrent GPU Ray actors, scheduling dependent stages with **ray.wait** so each starts as soon as its inputs are ready instead of running sequentially.
- Built the geometry-detection pipeline shared across sports: Canny edge detection with Hough-transform line fitting for tennis baselines and cricket creases, extended with pose-guided K-Means segmentation for the yoga mat.
- Built the asset-delivery and error-handling layer (per-environment S3 uploads, CMS API registration) backed by **24 pytest cases across 20 modules** mocking Ray, S3, and API dependencies.

Pronisi, Inc. | *Machine Learning Intern*

Jun. 2023 – May 2024

- Managed the ML data pipeline end to end: video collection, cleaning, annotation, and preprocessing.
- Built video processing APIs for anchor-frame synchronization and split-screen comparison of user and pro videos.
- Built real-time pose estimation (30 keypoints) and keypoint-based equipment detection for tennis, cricket, basketball, and golf using MediaPipe, YOLO, Faster R-CNN, ResNet, and SAM, also evaluating Detectron2, Mask R-CNN, and MobileNet.
- Reached **80%+ accuracy** tracking fast-moving balls and equipment with YOLOv7/8 and trajectory smoothing; served models with **ONNX Runtime** (40% faster inference) via GPU-enabled Docker (AWS ECS + CloudWatch).

PROJECTS

Kafka from Scratch | *Python, ZooKeeper, RabbitMQ, Redis*

<https://bit.ly/3T8GpCz>

- Built a Kafka-style pub/sub system in two versions (raw TCP sockets and RabbitMQ) with three brokers, ZooKeeper coordination, and Redis-backed metadata, as a team of four for a Big Data course.
- Implemented fault tolerance: leader election, heartbeat monitoring (5 s, with retries), and automatic failover, so clients re-resolve topic leaders and keep working with one broker left.
- Added key-hash partition routing (per-key ordering), 3-way active replication, and consumer offsets with read-from-beginning replay.

Microservices with RabbitMQ | *Python, Flask, RabbitMQ, MongoDB*

<https://bit.ly/457wShz>

- Built a Flask REST producer publishing to two RabbitMQ direct exchanges, routing health checks and CRUD requests by key to 4 dedicated consumer services in pub/sub and work-queue patterns.
- Used durable queues with manual ack for at-least-once delivery into MongoDB, and orchestrated the 6-container stack (producer, 4 consumers, MongoDB) with docker-compose across two custom networks.